Java OOP — Part 3.4: Interfaces in Depth

Part 3.4 covers **interfaces**: Java's tool for defining a contract that unrelated classes can fulfil. It builds on abstraction (3.2.3), inheritance (3.3.3) and polymorphism (3.3.5), and finishes the picture of how types relate in Java.

3.4.1 What an Interface Is, and the Keywords extends, implements and interface

An **interface** is a reference type that declares **what** a class can do, without saying **how**. It is a **contract**: any class that `implements` the interface promises to provide all of its abstract methods, and any code can then rely on that promise without knowing the class.

```
interface Payment {                          // the contract
    void pay(double amount);
    String name();
}

class UpiPayment implements Payment {       // a class fulfils the contract
    @Override public void pay(double amount) { System.out.println("UPI " +
amount); }
    @Override public String name() { return "UPI"; }
}
class CardPayment implements Payment {
    @Override public void pay(double amount) { System.out.println("Card " +
amount); }
    @Override public String name() { return "Card"; }
}

Payment p = new UpiPayment();               // variable of the interface type
p.pay(500);                                 // runs UpiPayment's code
(runtime polymorphism)
// Payment q = new Payment();               // ERROR: an interface cannot be
instantiated
```

What you can achieve with interfaces

Goal	How interfaces help	Real example
Abstraction	Callers depend on the contract, not on implementation details	`java.sql.Connection`: PostgreSQL, MySQL and H2 each implement it
Loose coupling (program to an interface)	Code uses `List`, so the implementation can change	`List<String> l = new ArrayList<>();` can become `LinkedList`
Multiple inheritance of type	A class can implement many interfaces, though it extends one class	`ArrayList` is a `List`, `RandomAccess`, `Cloneable`, `Serializable`
Runtime polymorphism	Different classes respond to the same call in their own way	A `Payment` list processed uniformly
Capabilities across unrelated classes	Any class can opt in to a role	`Comparable`, `Runnable`, `AutoCloseable`
Testability and dependency injection	Replace real collaborators with fakes or mocks	Spring injects a `UserRepository`; tests supply a fake
Callbacks and lambdas	A functional interface is the target type of a lambda	`Comparator`, `Runnable`, `Function`
Plug-in and API design	Publish only the contract; implementations stay replaceable	Servlet API, JDBC, `java.util.concurrent.Executor`
Evolving an API safely	`default` methods add behavior without breaking implementers	`List.sort`, `Collection.stream` added in Java 8

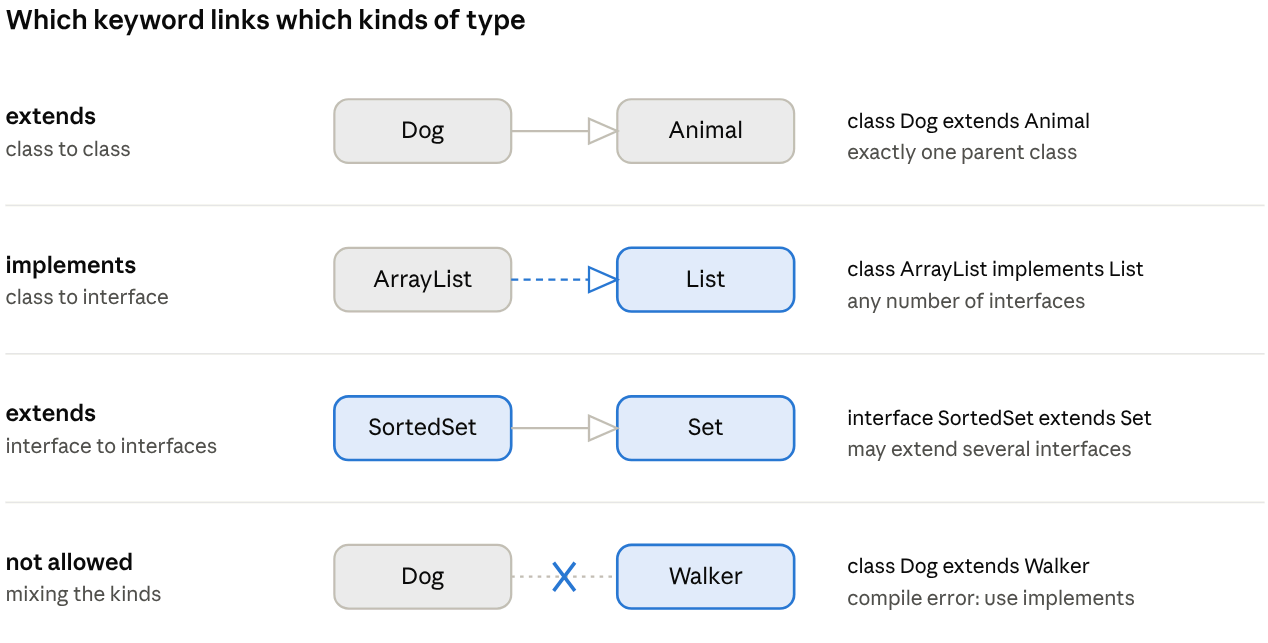
Declaring an interface

```
public interface Repository<T, ID> {        // may be generic
    T findById(ID id);                      // abstract by default: no body
    void save(T entity);
    default boolean exists(ID id) { return findById(id) != null; }   //
optional behavior
}
```

- A **top-level** interface is `public` or package-private. A **nested** interface may also be `protected` or `private`.
- The compiler produces a normal `.class` file. `javap` shows it as `interface` and `abstract`.
- An interface **cannot be instantiated**, has **no constructors** and **no instance fields**. It can be used as the type of variables, parameters and return values, and you can create objects of it only through an implementing class, an anonymous class or a lambda.
- A concrete class that `implements` an interface must **implement every abstract method**, or be declared `abstract` itself.
- Interface methods are **implicitly** `public`, so the implementing method must also be `public`. Forgetting that is a common error: `void pay(double a) { }` in the class fails with *attempting to assign weaker access privileges*.
- Interfaces do not extend `Object`, but you can call `toString()`, `equals()` and `hashCode()` on an interface reference, because every real object is an `Object`.

```
Payment p = new Payment() {                 // anonymous class implementing
the interface
    public void pay(double a) { System.out.println("test " + a); }
    public String name() { return "test"; }
};
if (p instanceof UpiPayment u) { /* test the real class, bind u */ }
```

The three keywords: extends, implements, interface



Shaded boxes are interfaces, plain boxes are classes. A solid line is extends, a dashed line is implements.
A class can do both at once: class Duck extends Bird implements Flyer, Swimmer.

extends, implements and interface: which links which

Relationship	Keyword	How many?	Example
Class to **class**	`extends`	**One** parent class	`class Dog extends Animal`
Class to **interface**	`implements`	**Any number**	`class Duck implements Flyer, Swimmer`
Interface to **interface**	`extends`	**Any number**	`interface Amphibious extends Flyer, Swimmer`
Interface to class	not allowed		An interface cannot extend or implement a class
Class to interface with `extends`	not allowed		`class Dog extends Walker` is an error

- `interface` is the keyword that **declares** the type.
- `implements` says a **class** *fulfils* a contract. `extends` between interfaces says one contract *builds on* others.

- A class can combine both, in this order: **`extends` first, then `implements`**.

```
class Duck extends Bird implements Flyer, Swimmer, Comparable<Duck> { /* ... */ }
interface Amphibious extends Flyer, Swimmer { }       // one contract made of two
class Frog implements Amphibious { /* must supply fly(), swim() */ }
```

extends vs implements, in one line: `extends` inherits **implementation and state** from a class (so only one is allowed); `implements` accepts a **contract** (so several are fine).

Interface compared with a class

	Class	Interface
Instantiate with `new`	Yes (if concrete)	No
Constructors	Yes	No
Instance fields (state)	Yes	No (constants only)
Method bodies	Yes	Only `default`, `static` and `private` methods
Inheritance	One parent class	Any number of parent interfaces
Member access	Any modifier	Methods `public` (private helpers since Java 9)
Purpose	An implementation	A contract or capability

Interview traps

- *Can an interface have a constructor?* No: it has no state to initialize and cannot be instantiated.
- *Can a class implement an interface without implementing all methods?* Only if the class is `abstract`.
- *Can an interface extend multiple interfaces?* Yes. *Can a class extend multiple classes?* No.

- *Does an interface extend `Object`?* Not formally, yet `Object`'s methods are callable on any interface reference.
- *Why must implementing methods be `public`?* Because interface methods are implicitly public and an override cannot reduce visibility.
- *Can we create an object of an interface type?* Only of an implementing class, an anonymous class or a lambda, never with `new Interface()` directly.

3.4.2 Interface Variables and Methods

Inside an interface, the compiler **adds modifiers for you**. Knowing the implicit rules, and the reasons behind them, is a standard interview topic.

Member	Implicit modifiers	Why
Variable (field)	`public static final`	A constant of the contract
Abstract method	`public abstract`	A promise that implementers must keep
`default` and `static` method	`public`	Part of the contract; has a body
`private` method (Java 9+)	`private` (explicit)	A helper for the interface's own default and static methods
Nested class, interface or enum	`public static`	Belongs to the contract and needs no instance
The interface itself	`abstract`	It cannot be instantiated

Interface variables are `public static final`

Every variable in an interface is a **constant**, whether or not you write the modifiers.

```
interface Config {
    int MAX_USERS = 100;                  // same as: public static final int
MAX_USERS = 100;
    String APP_NAME = "billing";          // same as: public static final
String APP_NAME = "billing";
    // int count;                         // ERROR: must be initialized (a
final needs a value)
    // private int secret = 1;            // ERROR: modifier private not
allowed here
    // protected int level = 2;           // ERROR: only public, static, final
are allowed
}

class Server implements Config {
    void start() {
        System.out.println(MAX_USERS);          // inherited by simple name
        System.out.println(Config.MAX_USERS);   // or through the interface
name
        // MAX_USERS = 200;                      // ERROR: cannot assign a
value to a final variable
    }
}
```

Why `public`? An interface is a contract meant to be seen by **every** implementer, wherever it lives (any package), and by every caller. A `private` or `protected` constant could not be shared with those parties, which would defeat its purpose.

Why `static`? An interface has **no instances and no state of its own**. A field must therefore belong to the **type**, with exactly one copy. If interfaces could declare instance fields, a class implementing several of them would inherit state from many parents, bringing back the diamond problem of multiple inheritance of state (3.3.3) that Java avoided by design.

Why `final`? The value is **shared by every implementer and every caller**. If one could reassign it, a change in one place would silently alter the contract everywhere else, which is global mutable state. A contract value must not change, so it is fixed once.

Details

- The initializer is mandatory and may be any expression, not only a literal. A literal or constant expression of a primitive or `String` type is a **compile-time constant** and is inlined into callers (2.6). A value such as `List<String> ROLES = List.of("A",`

`"B");` is evaluated at run time, when the interface is initialized.

- The **interface is initialized lazily**, on first access to a non-constant field. Initializing a *class* does not initialize its interfaces, **except** interfaces that declare `default` methods, which are initialized when the implementing class is.
- The name is inherited, so two interfaces with the same constant name make the simple name **ambiguous** in an implementing class (see 3.4.4).
- A `final` constant of a **mutable type** is still mutable inside: `List<String> ROLES = new ArrayList<>();` can have elements added by anyone. Use `List.of(...)` or another immutable value.

The "constant interface" anti-pattern. Implementing an interface only to get unqualified access to its constants (`class Server implements Config`) leaks them into the class's own exported API, and they stay there forever. Use instead:

```
public final class Limits {                    // a non-instantiable holder
    private Limits() { }
    public static final int MAX_USERS = 100;
}
// import static com.example.Limits.MAX_USERS;  then use MAX_USERS directly

enum Level { LOW, MEDIUM, HIGH }               // for a fixed set of related
named values
```

Constants that genuinely belong to a contract (for example the allowed states of an `Order` API) can stay in the interface, referenced as `Order.STATUS_NEW`.

Interface methods are `public abstract`

A method without a body in an interface is **abstract** and **public** automatically.

```
interface Shape {
    double area();                  // same as: public abstract double
area();
    public abstract double perimeter();   // legal, but the modifiers are
redundant (style guides say omit them)
}
```

Why `public`? The method is the contract that callers use through the interface type, from any package. Implementers must expose it equally widely. Since an overriding method cannot reduce visibility (3.3.4), the implementing method must be `public` as well.

```
class Circle implements Shape {
    double r;
    // double area() { return Math.PI * r * r; }       // ERROR: attempting
to assign weaker access privileges; was public
    @Override public double area() { return Math.PI * r * r; }   // OK
    @Override public double perimeter() { return 2 * Math.PI * r; }
}
```

Why `abstract`? An interface says **what**; the **how** belongs to the implementing classes. A method with no body is, by definition, abstract. This also gives the freedom for **many unrelated implementations** of the same contract.

Modifiers that are not allowed on an abstract interface method

Modifier	Why it is rejected
`private`, `protected`, package-private	Would hide the method from implementers or callers; the contract must be public
`final`	Final forbids overriding, but an abstract method exists only to be implemented
`static`	Static methods are not overridden, so they cannot be abstract (static methods with a body are allowed since Java 8)
`synchronized`, `native`, `strictfp`	These describe an implementation, and an abstract method has none

```
interface Bad {
    // protected void a();      // ERROR
    // final void b();          // ERROR
    // void c() { }             // ERROR: interface abstract methods cannot
have a body (unless default or static)
    // private void d();        // ERROR: a private interface method needs a
body
}
```

Seeing it in the bytecode

```
$ javap -p Config
interface Config {
  public static final int MAX_USERS = 100;
  public static final java.lang.String APP_NAME;
}
$ javap -p Shape
interface Shape {
  public abstract double area();
  public abstract double perimeter();
}
```

The compiler has written the implicit modifiers into the class file. Through reflection, `Modifier.toString(method.getModifiers())` returns `public abstract` for such a method.

Why interfaces have no instance fields

To recap the design: no instance fields means no state to initialize (so no constructors), no duplicated state when many interfaces are combined, and implementing classes stay free to extend any one class that carries state. Behavior can be shared with `default` methods, which can call only the interface's other methods, not fields (3.4.3).

Interview traps

- *What modifiers does the compiler add to an interface field?* `public static final`.
- *Can an interface field be private or non-final?* No; both are compile errors.
- *Why are interface fields `static` and `final`?* No instances to hold state, and shared contract values must not change.
- *What are the implicit modifiers of an abstract method in an interface?* `public abstract`.
- *Can an implementing method have default (package) access?* No: it would reduce visibility, which is a compile error.
- *Can an interface method be `final`?* No (a `default` method also cannot be `final`, so implementers can always override it).
- *Is it good practice to put constants in an interface?* Only for constants that are part of the contract; otherwise use a final holder class, an enum, or static imports.

3.4.3 Kinds of Interface Members: Abstract, Constant, Default, Static and Private

Interfaces grew over time. Early Java allowed only constants and abstract methods; Java 8 and 9 added methods **with bodies**, to solve concrete problems.

Java version	What interfaces gained	Why
1.0 to 7	Constants and abstract methods, nested types	Pure contract
8 (2014)	`default` and `static` methods	Evolve old interfaces (add `stream()` to `Collection`) and enable lambdas and streams
9 (2017)	`private` and `private static` methods	Share code between default and static methods without exposing it
17	`sealed` interfaces	Restrict who may implement (3.4.5)

1. Abstract methods: the contract

A method with **no body** is abstract and public. It is what every implementer must provide.

```
interface Notifier {
    void send(String to, String message);       // the contract
}
class EmailNotifier implements Notifier { public void send(String to, String
m) { /* SMTP */ } }
class SmsNotifier   implements Notifier { public void send(String to, String
m) { /* gateway */ } }
```

Usage and benefits: callers use `Notifier` and do not care which one they get (polymorphism); new implementations can be added without changing callers; the contract is the unit you test against and mock.

2. Constant variables

`public static final` values that belong to the contract (3.4.2).

```
interface HttpStatusCodes { int OK = 200; int NOT_FOUND = 404; }
```

Usage and benefits: named values shared by all implementers and callers, with no instance needed. Keep them to values that are truly part of the contract.

3. Default methods (Java 8)

A `default` method has a **body** inside the interface. Implementing classes **inherit** it and may **override** it.

```
interface Validator<T> {
    boolean validate(T t);                                          //
abstract: the one thing implementers supply

    default Validator<T> and(Validator<T> other) {                  //
behavior built from the abstract method
        return t -> this.validate(t) && other.validate(t);
    }
    default Validator<T> negate() { return t -> !this.validate(t); }
}

Validator<String> notEmpty = s -> !s.isEmpty();
Validator<String> shortText = s -> s.length() < 20;
Validator<String> ok = notEmpty.and(shortText);                      //
composition without any extra class
System.out.println(ok.validate("hello"));                            // true
```

Why default methods exist: evolving APIs. Before Java 8, adding a method to a published interface broke **every** class implementing it (they would no longer compile). To add `stream()`, `forEach` and `sort` to `Collection`, `Iterable` and `List` without breaking millions of implementations, the designers let interfaces provide a default body. That is how `List.sort(Comparator)`, `Collection.stream()`, `Map.getOrDefault(...)` and `Iterable.forEach(...)` arrived.

Other uses and benefits

- **Fluent combinators** on functional interfaces: `Comparator.reversed()` and `thenComparing(...)`, `Predicate.and/or/negate`, `Function.andThen`.
- **Optional or convenience behavior:** a default that most implementers accept, overridden by a few.
- **Reduced boilerplate** compared with an abstract skeleton class, and a limited form of multiple inheritance of **behavior**.

Rules for default methods

- Declared with the `default` keyword and a body; implicitly `public`.
- Inherited by implementing classes and sub-interfaces; a class **may override** it. A sub-interface may **override**, or **re-declare it abstract**, forcing implementers to supply it again.
- Cannot be `static`, `final`, `abstract` or `synchronized`. It cannot be `final` precisely so that implementers can always override it.
- It can call the interface's abstract and other default methods, using `this`, but it has **no instance fields**, so no state.
- It **cannot override `Object` methods**: `default boolean equals(Object o)`, `hashCode()` and `toString()` are compile errors, because a class's `Object` methods always take precedence.
- To call a specific parent's default from an implementing class: `InterfaceName.super.method()`.

```
interface Walker { default String move() { return "walk"; } }
class Robot implements Walker {
    @Override public String move() { return Walker.super.move() + " on
wheels"; }   // reuse and extend
}
```

Cautions. Do not use defaults to smuggle a full implementation that really needs state; use an abstract class or composition. Adding a default method to an existing interface can still clash with a **method already present in an implementing class** (the class method wins, but if signatures differ in return type, compilation fails). Huge defaults make the interface hard to read.

4. Static methods (Java 8)

A `static` method belongs to the **interface itself**.

```
interface Payment {
    void pay(double amount);

    static Payment of(String type) {                        // a factory kept
next to the type it creates
        return switch (type) {
            case "upi"  -> a -> System.out.println("UPI " + a);
            case "card" -> a -> System.out.println("Card " + a);
            default -> throw new IllegalArgumentException("unknown: " +
type);
        };
    }
}
Payment.of("upi").pay(100);                                 // called on the
interface name only
```

Rules

- Called as `InterfaceName.method()`. It is **not inherited**: not by implementing classes, not by sub-interfaces. `new UpiPayment().of("x")` and `UpiPayment.of("x")` do not compile.
- It cannot be overridden and cannot be `abstract` or `default`. It can be `public` (the default) or `private`.
- It may call other static or private methods of the interface, but not instance methods without an object.

Usage and benefits: helper and factory methods live **next to** the interface instead of in a separate utility class (`Collections` for `Collection`, `Arrays` for arrays). Examples: `List.of(...)`, `Map.entry(...)`, `Comparator.naturalOrder()`, `Stream.of(...)`, `Path.of(...)`. Keeping them in the interface improves discoverability and keeps the API in one place.

5. Private methods (Java 9)

A `private` method (instance or `static`) has a body and is visible **only inside the interface**.

```
interface Logger {
    void write(String line);

    default void info(String m)  { write(format("INFO", m)); }
    default void warn(String m)  { write(format("WARN", m)); }
    static Logger console()      { return line -> System.out.println(stamp()
+ line); }

    private String format(String level, String m) { return "[" + level + "] "
+ m; }   // shared helper, hidden
    private static String stamp() { return java.time.Instant.now() + " "; }
// shared static helper
}
```

Usage and benefits: removes **duplicated code** between several default or static methods, and keeps the helpers out of the public API, so implementers and callers cannot see or depend on them. A private instance method may call other instance methods of the interface; a private static method cannot. Private methods cannot be `abstract` and cannot be `default`.

The five kinds side by side

Kind	Keyword	Body?	Visibility	Inherited by implementers?	Overridable?	Since
Abstract method	none	No	public	Must be implemented	Implemented	1.0
Constant	none	Initializer	public	Yes (name)	No (final)	1.0
Default method	`default`	Yes	public	**Yes**	**Yes**	8
Static method	`static`	Yes	public or private	**No**	No	8
Private method	`private`	Yes	private	No	No	9

Evolving an interface safely

```
// Version 1 of a published library interface
interface Greeter { String greet(String name); }

// Version 2 adds behavior WITHOUT breaking existing implementers:
interface Greeter {
    String greet(String name);
    default List<String> greetAll(List<String> names) {        // new in v2
        return names.stream().map(this::greet).toList();
    }
}
```

Before Java 8, adding `greetAll` as an abstract method would have forced every implementer to change.

Interview traps

- *Why were default methods introduced?* To add methods to existing interfaces (such as `Collection.stream()`) without breaking implementers, and to support lambdas and streams.
- *Are static interface methods inherited?* No; call them through the interface name.
- *Can a default method be `final`?* No.
- *Can an interface define a default `toString()`?* No, a compile error: `Object`'s methods cannot be defaulted.
- *Can an interface contain `main`?* Yes: `interface App { static void main(String[] a) { ... } }` runs like a class.
- *Difference between an interface with default methods and an abstract class?* Defaults cannot hold state, cannot be `final`, are always public, and a class can implement many interfaces but extend only one class.
- *What are private interface methods for?* Sharing code between default and static methods without exposing it.

3.4.4 Extending Interfaces, Multiple Interfaces, Polymorphism and Naming Conflicts

Extending an interface

An interface can **extend one or more interfaces**. It inherits all their abstract and default methods and constants, and may add new members, override default methods, or re-declare them abstract.

```
interface ReadRepository<T, ID> {
    T findById(ID id);
    List<T> findAll();
}
interface WriteRepository<T, ID> {
    void save(T entity);
    void deleteById(ID id);
}
interface CrudRepository<T, ID> extends ReadRepository<T, ID>,
WriteRepository<T, ID> {   // combines two contracts
    default boolean exists(ID id) { return findById(id) != null; }
}

class UserRepository implements CrudRepository<User, Long> {
    // must implement findById, findAll, save, deleteById (all inherited
abstract methods)
}
```

Splitting a large contract into **small, focused interfaces** and combining them where needed follows the **Interface Segregation Principle**: clients should not depend on methods they do not use. A reporting service can then accept a `ReadRepository` and cannot accidentally delete data. JDK examples: `List extends SequencedCollection` (Java 21) which extends `Collection`, which extends `Iterable`; `Deque extends Queue`.

Rules

- Use `extends` (never `implements`) between interfaces; list several with commas.
- A class that implements the sub-interface must implement **all** abstract methods from the whole chain, or be `abstract`.
- Cyclic interface inheritance is a compile error.
- A sub-interface can **override** an inherited default method with its own body, or **re-declare** it without a body to turn it back into an abstract method.

Implementing multiple interfaces

A class may implement any number of interfaces. This is how Java provides **multiple inheritance of type** (3.3.3).

```
interface Flyer   { void fly(); }
interface Swimmer { void swim(); }
interface Walker  { void walk(); }

class Duck implements Flyer, Swimmer, Walker {
    public void fly()  { System.out.println("flap"); }
    public void swim() { System.out.println("paddle"); }
    public void walk() { System.out.println("waddle"); }
}

Duck d = new Duck();
Flyer f = d;        // each interface is a different VIEW of the same object
Swimmer s = d;
Walker w = d;
```

The same object can be used wherever **any** of its types is expected. `Flyer f` can call only `fly()`, even though the object can also swim, because the reference type decides which methods are visible.

Polymorphism through interfaces

Interface types are the main vehicle for **runtime polymorphism** (3.3.5). A reference of an interface type can hold any object whose class implements it, and a call is dispatched to the real class.

```
interface Notifier { void send(String to, String msg); }
class EmailNotifier implements Notifier { public void send(String to, String
m) { System.out.println("mail " + to); } }
class SmsNotifier   implements Notifier { public void send(String to, String
m) { System.out.println("sms " + to); } }
class PushNotifier  implements Notifier { public void send(String to, String
m) { System.out.println("push " + to); } }

class AlertService {
    private final List<Notifier> channels;                      // depends
only on the contract
    AlertService(List<Notifier> channels) { this.channels = channels; }
    void alert(String to, String msg) { channels.forEach(n -> n.send(to,
msg)); }   // one call, many behaviors
}

new AlertService(List.of(new EmailNotifier(), new
SmsNotifier())).alert("asha", "disk full");
```

Adding `PushNotifier` requires no change to `AlertService`. In tests you pass a recording fake. In Spring you declare `List<Notifier>` and the container injects **every** bean implementing the interface.

```
Object o = new Duck();
if (o instanceof Flyer f) f.fly();            // test for a capability, not a
concrete class
Flyer[] flock = { new Duck(), () -> System.out.println("glide") };   // a
lambda is a Flyer too
```

Conversions: upcasting to an implemented interface is implicit and always safe; downcasting needs a cast and can throw `ClassCastException`. Casting to an interface is allowed even when the compile-time class does not implement it (a subclass might), unless the class is `final` and does not.

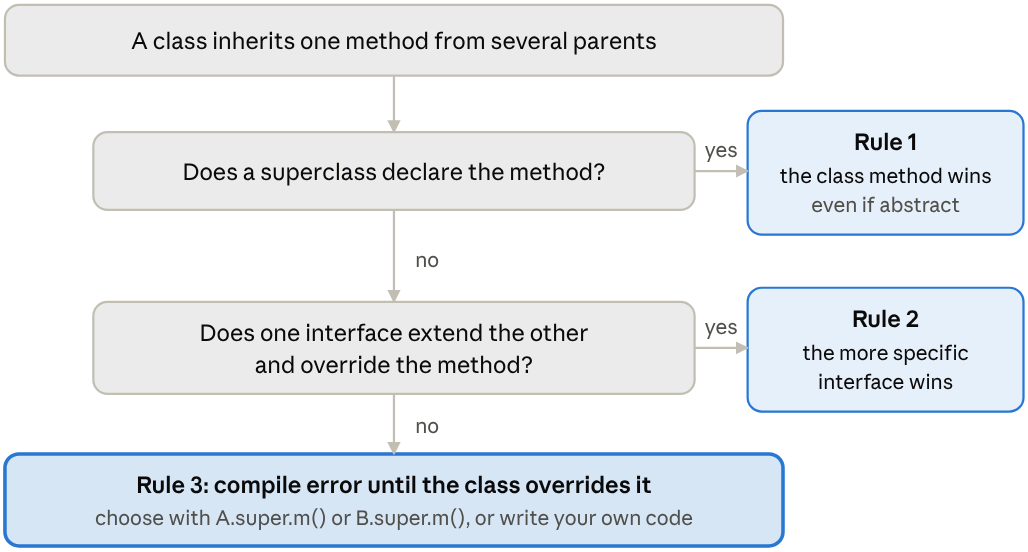
Naming conflicts

Because a class or interface may inherit from several interfaces, the **same name** can arrive from more than one place. Java resolves each case with fixed rules.

Situation	Result
Two interfaces declare the **same abstract method** (same signature and return type)	No conflict: **one implementation** satisfies both
Same signature, **incompatible return types** (for example `int` and `String`)	**Compile error**: no class can implement both. A return type that is a subtype of the other is fine (covariant)
Same name, **different parameters**	Overloads: implement both
Two unrelated **default** methods, same signature	**Compile error** until the class overrides it (Rule 3 below)
A **default** in one interface and an **abstract** method in another, same signature	**Compile error** until the class implements it
A **class method** (declared or inherited) and an interface default	The **class method wins** (Rule 1)
Default in `A`, and `B extends A` overrides it	`B`'s version wins when a class implements both (Rule 2)
Static methods with the same name in two interfaces	No conflict: static interface methods are **not inherited**
Same **constant** name in two interfaces	A simple-name use is **ambiguous**: qualify it (`A.LIMIT`, `B.LIMIT`)
A default method that matches an `Object` method (`equals`, `hashCode`, `toString`)	**Compile error** in the interface itself

The three rules for conflicting methods (this is the interview answer):

Which implementation does a class get? Apply the rules in order



Resolving a method inherited from several parents

```
// Rule 3: unrelated defaults clash, and the class must decide
interface Camera { default String describe() { return "camera"; } }
interface Phone  { default String describe() { return "phone"; } }

class SmartPhone implements Camera, Phone {
    @Override public String describe() {
        return Camera.super.describe() + " + " + Phone.super.describe();
// pick, combine, or write your own
    }
}
System.out.println(new SmartPhone().describe());     // camera + phone

// Rule 1: the class wins, even over a default
class Base { public String describe() { return "base"; } }
class Gadget extends Base implements Camera { }       // no conflict
System.out.println(new Gadget().describe());          // base

// Rule 2: the more specific interface wins
interface Animal { default String sound() { return "..."; } }
interface Dog extends Animal { default String sound() { return "woof"; } }
class Puppy implements Animal, Dog { }                // Dog is more specific
System.out.println(new Puppy().sound());              // woof
```

Details of `X.super.m()`: `X` must be a **direct** superinterface of the class (or interface) you are writing; you cannot reach a grandparent's default this way. It is legal only in a class or interface body that inherits `X`'s method.

Constant clash

```
interface A { int LIMIT = 10; }
interface B { int LIMIT = 20; }
class C implements A, B {
    void show() {
        // System.out.println(LIMIT);          // ERROR: reference to LIMIT
is ambiguous
        System.out.println(A.LIMIT + " " + B.LIMIT);   // OK: 10 20
    }
}
```

Return-type clash

```
interface X { int  value(); }
interface Y { long value(); }
// class Z implements X, Y { }      // ERROR: no single value() can return
both int and long
```

Abstract meets default

```
interface P { default void run() { System.out.println("P"); } }
interface Q { void run(); }                          // abstract
class R implements P, Q {
    @Override public void run() { P.super.run(); }   // required: the class
must implement it
}
```

Interview traps

- *Can an interface extend multiple interfaces? Can a class implement multiple interfaces?* Yes to both.
- *What happens if two interfaces have the same default method?* The class must override it (or inherit a class method that wins); use `A.super.m()` to reuse one.
- *Which wins, a class method or an interface default?* The class method, always.
- *Are static interface methods a conflict source?* No, they are not inherited.

- *How do you disambiguate two constants with the same name?* Qualify with the interface name.
- *Can an interface extend a class?* No.
- *Is the diamond problem possible in Java?* Not for state. For behavior it can occur with defaults, and the compiler forces a resolution.

3.4.5 Types of Interfaces

The language has one `interface` construct, but interfaces are used in several well-known **roles**, which is what interviewers mean by "types of interfaces".

Type	Defining feature	Examples
Normal (ordinary, service) interface	Several abstract methods forming a contract	`List`, `Connection`, `UserService`
Marker (tagging) interface	**No members at all**	`Serializable`, `Cloneable`, `RandomAccess`
Single Abstract Method (SAM) interface	Exactly **one abstract method**	`Runnable`, `Comparable`, `Callable`
Functional interface	A SAM interface meant for lambdas, usually `@FunctionalInterface`	`Function`, `Predicate`, `Comparator`
Sealed interface	Only **permitted** types may implement it	`sealed interface Shape permits ...`
Generic interface	Declared with type parameters	`Comparable<T>`, `Repository<T, ID>`
Nested interface	Declared inside a class or interface	`Map.Entry`, `Thread.UncaughtExceptionHandler`
Annotation interface	Declared with `@interface`	`@Override`, `@FunctionalInterface`

Type	Defining feature	Examples
Constant interface (anti-pattern)	Only constants	Avoid (3.4.2)

Normal interfaces

An ordinary interface with a coherent set of methods, defining a service or capability. Keep it **small and cohesive** (Interface Segregation Principle).

```
public interface UserService {
    User register(String email, String password);
    Optional<User> find(long id);
    void deactivate(long id);
}
```

Marker interfaces

A **marker interface** is an **empty** interface used to **tag** a class with a property. The tag is checked at runtime with `instanceof`, or by library code, never through a method call.

```
public interface Serializable { }      // java.io: "this object may be
written to a stream"
public interface Cloneable { }         // java.lang: "Object.clone() may copy
this object"
public interface RandomAccess { }      // java.util: "get(i) is fast", so
algorithms may index
```

How a marker takes effect

```
class Report implements Serializable { }
class Secret { }

new ObjectOutputStream(out).writeObject(new Report());   // fine
new ObjectOutputStream(out).writeObject(new Secret());   //
NotSerializableException: the runtime checks 'instanceof Serializable'

// Collections.binarySearch(list, key) checks 'list instanceof RandomAccess'
to choose index-based or iterator-based search
```

A custom marker

```
interface Auditable { }                                    // marker: changes
to these entities must be logged
class Invoice implements Auditable { }

void save(Object entity) {
    if (entity instanceof Auditable) audit.log(entity);   // behavior depends
on the tag
}
void archive(Auditable a) { }                               // a marker is
also a TYPE: parameters can require it
```

Marker interface vs annotation

	Marker interface	Marker annotation (`@Entity` , `@Deprecated`)
Defines a **type**	**Yes**: usable as a parameter or variable type, so misuse is caught at compile time	No
Can apply to	Classes only (those that implement it)	Classes, methods, fields, parameters, packages
Can carry data	No	Yes, through elements (`@Table(name = "users")`)
Checked by	`instanceof`	Reflection or annotation processing
Inherited by subclasses	Yes, automatically	Only with `@Inherited` , and only for classes

Guideline (*Effective Java*): use a **marker interface** when you want a type you can use to restrict parameters or collections; use an **annotation** when the mark applies to program elements other than classes or needs parameters.

Single Abstract Method (SAM) interfaces

A SAM interface has **exactly one abstract method**. It may have any number of `default` , `static` and `private` methods and constants.

```
interface Greeter { String greet(String name); }          // one abstract
method

Greeter g1 = new Greeter() { public String greet(String n) { return "Hi " +
n; } };   // anonymous class
Greeter g2 = n -> "Hi " + n;                               // lambda: only
legal for SAM interfaces
Greeter g3 = String::toUpperCase;                          // method
reference
```

JDK SAM examples: `Runnable.run()`, `Callable.call()`, `Comparable.compareTo(T)`, `AutoCloseable.close()`, `Iterable.iterator()`.

Functional interfaces

A **functional interface** is a SAM interface intended to be the **target type of a lambda or method reference**. The optional annotation `@FunctionalInterface` asks the compiler to **verify** that there is exactly one abstract method, and flags an error when someone later adds a second.

```
@FunctionalInterface
interface TriFunction<A, B, C, R> {
    R apply(A a, B b, C c);
    default <V> TriFunction<A, B, C, V> andThen(Function<? super R, ? extends
V> after) {
        return (a, b, c) -> after.apply(apply(a, b, c));
    }
}
TriFunction<Integer, Integer, Integer, Integer> sum3 = (x, y, z) -> x + y +
z;
```

What counts as the single abstract method

- Methods inherited from `Object` that are re-declared abstract (`boolean equals(Object)`) **do not count**. That is why `Comparator` is functional even though it lists `equals`: its only other abstract method is `compare`.
- `default` and `static` methods do not count.
- A functional interface may **extend** another, as long as the total abstract methods remain one.

The built-in toolkit (`java.util.function`, Java 8+):

Interface	Method	Meaning
`Function<T,R>`	`R apply(T t)`	Transform
`BiFunction<T,U,R>`	`R apply(T t, U u)`	Transform two inputs
`Predicate<T>`	`boolean test(T t)`	Test a condition
`Consumer<T>`	`void accept(T t)`	Use a value, return nothing
`Supplier<T>`	`T get()`	Produce a value
`UnaryOperator<T>`	`T apply(T t)`	Function from `T` to `T`
`BinaryOperator<T>`	`T apply(T a, T b)`	Combine two `T` into one
`IntFunction`, `ToIntFunction`, `IntPredicate`…	primitive forms	Avoid boxing

`Runnable`, `Callable<V>`, `Comparator<T>` and `Comparable<T>` are older functional interfaces that predate the package. Functional interfaces with their combinators (`Predicate.and`, `Function.andThen`, `Comparator.thenComparing`) are the base of the Streams API.

Generic interfaces

```
interface Converter<S, T> { T convert(S source); }
class StringToInt implements Converter<String, Integer> {
    @Override public Integer convert(String s) { return Integer.parseInt(s);
}
}
interface Repository<T, ID> { Optional<T> findById(ID id); }       // JPA /
Spring Data style
```

Implementers fix the type arguments, or stay generic themselves (`class Cache<K, V> implements Repository<V, K>`). A class cannot implement the same generic interface with two different type arguments (`Comparable<A>` and `Comparable<B>`), because of type erasure.

Nested interfaces

An interface declared **inside** a class or another interface is implicitly `static`.

```
class Button {
    interface ClickListener { void onClick(Button b); }    // nested: scoped
to Button
    void addListener(ClickListener l) { /* ... */ }
}
button.addListener(b -> System.out.println("clicked"));    //
Button.ClickListener is a SAM, so a lambda works
```

JDK examples: `Map.Entry<K,V>`, `Thread.UncaughtExceptionHandler`. Nested interfaces keep a closely related contract next to the class that uses it.

Sealed interfaces (Java 17)

A **sealed** interface lists exactly which types may implement or extend it. Every permitted subtype must be `final`, `sealed` or `non-sealed`; records are implicitly final.

```
sealed interface Shape permits Circle, Rectangle, Triangle { }

record Circle(double r) implements Shape { }
record Rectangle(double w, double h) implements Shape { }
final class Triangle implements Shape { final double b, h; Triangle(double b,
double h) { this.b = b; this.h = h; } }

static double area(Shape s) {
    return switch (s) {                                        // exhaustive:
no default needed
        case Circle c    -> Math.PI * c.r() * c.r();
        case Rectangle r -> r.w() * r.h();
        case Triangle t  -> 0.5 * t.b * t.h;
    };                                                          // adding a
fourth Shape makes this a compile error
}
```

Sealed interfaces model **closed families** (the "sum types" of functional languages): a `Result` that is `Success` or `Failure`, the nodes of a syntax tree, domain events. The permitted classes must be in the same module (or same package in an unnamed module). Use open interfaces for extension points that outsiders should implement.

Annotation interfaces

Declared with `@interface`; they are special interfaces whose methods are the annotation's **elements**.

```
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
@interface Timed { String name() default ""; int warnMs() default 500; }

@Timed(name = "checkout", warnMs = 200)
void checkout() { }
```

Frameworks read them through reflection (`@Transactional`, `@GetMapping`, `@Entity`).

Other roles worth knowing

- **Capability (mixin) interfaces:** `Comparable`, `Closeable`, `Iterable` add one ability to any class.
- **Callback and listener interfaces:** `ActionListener`, `CompletionHandler`; usually SAM.
- **Service Provider Interfaces (SPI):** a contract that third parties implement and the JDK or framework discovers with `ServiceLoader` (JDBC drivers, logging backends).
- **Enums and records can implement interfaces** (they cannot extend classes), a common way to give them shared behavior.

Interview traps

- *What is a marker interface? Name three.* An empty interface used to tag classes: `Serializable`, `Cloneable`, `RandomAccess`.
- *How does `Serializable` work if it has no methods?* The serialization runtime checks `instanceof Serializable` and throws `NotSerializableException` if it fails.
- *Marker interface or annotation?* Interface when you need a compile-time type; annotation when you need metadata or to mark other elements.
- *What is a functional interface? Is `@FunctionalInterface` mandatory?* A SAM interface usable as a lambda target; the annotation is optional but recommended.
- *Can a functional interface have default methods?* Yes, any number. *Can it have `equals`?* Re-declaring `Object` methods does not count as abstract.
- *Is `Comparator` a functional interface?* Yes: one abstract method (`compare`); the rest are defaults, statics or `Object` methods.

- *Why sealed interfaces?* To close a hierarchy so the compiler can check `switch` exhaustiveness.

3.4.6 Abstract Class vs Interface, and How to Design with Interfaces

Both abstract classes (3.2.3) and interfaces describe incomplete types that others complete. Since Java 8 the line is blurrier, so you must know the real differences and a way to choose.

Side by side

	Abstract class	Interface
Relationship it models	**IS-A**: a family of related types	**CAN-DO** / contract: a capability that unrelated types can share
How a class uses it	`extends` (only **one**)	`implements` (**any number**)
Instance fields (**state**)	Yes, any kind	**No**; only `public static final` constants
Constructors	Yes (called through `super(...)`)	No
Method bodies	Any mix of abstract and concrete	`default`, `static`, `private` only
Member access levels	`public`, `protected`, package, `private`	Methods are `public` (private helpers since 9); fields `public`
Can a member be `final`?	Yes	No (not on default methods)
Lambda target	**No**	**Yes** (functional interfaces)
`sealed`, records, enums can use it	Records and enums cannot extend classes	Records and enums can implement interfaces
Evolve by adding a method	Add a concrete method safely	Add a `default` method safely (an abstract one breaks implementers)

	Abstract class	Interface
Protected hooks for subclasses	Yes	No
Can be combined	A class can extend one and implement many	

Common misconception: "an interface gives 100% abstraction and an abstract class 0 to 100%." That was true before Java 8. With default, static and private methods, an interface can contain implementation too. The reliable differences are **state, constructors, access control, and single vs multiple inheritance**.

Choosing between them

Ask these questions in order:

1. **Do unrelated classes need to share the capability?** (`Comparable`, `Closeable`, `Runnable`) Use an **interface**.
2. **Do you need a lambda target?** Use a **functional interface**.
3. **Must implementers be able to extend another class, or implement several contracts?** Use an **interface**.
4. **Do the implementers share state, need constructors that enforce invariants, or need `protected` hooks or `final` template methods?** Use an **abstract class**.
5. **Not sure?** Start with an **interface** for the contract, and add an **abstract skeletal class** only if the shared implementation pays for itself.

The recommended combination: interface plus skeletal implementation

Define the **type** as an interface, and offer an `AbstractXxx` class that implements most of it. Implementers choose to extend the skeleton or to implement the interface from scratch.

```
interface Shape {                                          // the contract: the
type everyone depends on
    double area();
    double perimeter();
    String describe();
}
abstract class AbstractShape implements Shape {          // skeleton: shared
code, optional to use
    private final String name;
    protected AbstractShape(String name) { this.name = name; }
    @Override public String describe() {                 // implemented in
terms of the primitives
        return name + ": area=" + String.format("%.2f", area()) + ",
perimeter=" + String.format("%.2f", perimeter());
    }
}
class Circle extends AbstractShape {                     // extends the
skeleton: writes only what is unique
    private final double r;
    Circle(double r) { super("Circle"); this.r = r; }
    public double area()      { return Math.PI * r * r; }
    public double perimeter() { return 2 * Math.PI * r; }
}
```

The JDK uses this pattern widely: `List` with `AbstractList`, `Map` with `AbstractMap`, `Collection` with `AbstractCollection`, `Set` with `AbstractSet`.

Some important JDK types are abstract **classes** rather than interfaces because they carry shared state or predate interfaces: `InputStream` and `OutputStream`, `Number`, `Reader` and `Writer`, `HttpServlet`.

Design guidance for interfaces

- **Program to an interface, not an implementation.** Declare variables, parameters and return types as `List`, `Map`, `Notifier`, and write the concrete class only after `new`.
- **Keep interfaces small and focused** (Interface Segregation). Several narrow interfaces combined by `extends` beat one "god interface".
- **Do not create an interface for every class.** One interface with a single implementation and no need for substitution is noise. Introduce one when there is a second implementation, a boundary to mock, or a plug-in point.

- **Name by role, not by implementation.** `PaymentGateway`, `Sortable`, `Closeable`, not `IPaymentGateway` or `PaymentGatewayImpl` (the `Impl` suffix usually signals a missing real name, like `RazorpayGateway`).
- **Treat a published interface as frozen.** Adding an abstract method breaks every implementer. Add a `default` method instead, or a new sub-interface.
- **Document the contract**: preconditions, postconditions, thread-safety and the exceptions each method may throw. Implementers and tests rely on it (3.3.6).
- **Use defaults sparingly**, for convenience or evolution, never for hidden state or large algorithms.
- **Separate API from implementation** by package or module: interfaces in an exported package, implementations in an internal one.
- **Optional operations are a design smell.** `List.add` on an immutable list throws `UnsupportedOperationException`, which is a known tension with substitutability; prefer separate types (`ReadOnlyList`) when you can.

Dependency inversion and injection

The **Dependency Inversion Principle** says high-level policy should depend on **abstractions**, not on low-level details. Interfaces are the abstraction, and **dependency injection** supplies the implementation.

```
@Service
class OrderService {
    private final PaymentGateway gateway;                  // depends on the
interface only
    OrderService(PaymentGateway gateway) { this.gateway = gateway; }   //
constructor injection
    void checkout(Order o) { gateway.charge(o.total()); }
}

// production: Spring injects RazorpayGateway;  test: new OrderService(new
FakeGateway())
```

With several implementations present, Spring needs a hint (`@Primary`, `@Qualifier`) or you inject a `List<PaymentGateway>` or `Map<String, PaymentGateway>` of all of them. Mocking libraries (Mockito) create test doubles most easily from interfaces.

Patterns that rest on interfaces

Pattern	Role of the interface	Example
Strategy	Interchangeable algorithms	`Comparator`, a `DiscountPolicy`
Observer / Listener	Callback contract	`ActionListener`, Spring `ApplicationListener`
Factory	Static method creates implementations	`List.of`, `Payment.of(type)`
Decorator	Wrapper implements the same interface	`BufferedReader` wraps a `Reader`
Adapter	Makes one interface look like another	`Arrays.asList`
Command	A single-method action	`Runnable`, `Callable`
Repository / DAO	Persistence behind a contract	Spring Data `JpaRepository`
Template Method	Usually an abstract class	`HttpServlet`

```
// Strategy with a functional interface: behavior as data
interface DiscountPolicy { double apply(double total); }
Map<String, DiscountPolicy> policies = Map.of(
    "none",     t -> t,
    "festival", t -> t * 0.9,
    "bulk",     t -> t > 10_000 ? t - 500 : t);
double payable = policies.get("festival").apply(2_000);      // 1800.0
```

Interview traps

- *Abstract class or interface after Java 8?* Interface for contracts, capabilities and lambdas; abstract class when sharing state, constructors, or protected hooks. Often both: interface plus skeletal abstract class.
- *Can an abstract class implement an interface without implementing its methods?* Yes; the methods stay abstract for the first concrete subclass.
- *Can an interface have a `main` method or be run?* Yes, a `static main` runs.

- *Is it true that interfaces provide 100% abstraction?* Not since Java 8, because of default and static methods.
- *Why do Spring and frameworks like interfaces?* They allow substitution, testing, and JDK dynamic proxies (Spring Boot defaults to CGLIB class proxies, but interfaces remain the better design boundary).
- *Which can a lambda target?* Only a functional interface, never an abstract class.

Review Questions: Part 3.4

Attempt every question first. Part A is multiple choice, Part B is short answer, Part C is coding and prediction, and Part D is senior level. All answers and explanations are consolidated in the next section.

Part A: Multiple Choice

Q1. What modifiers does the compiler add to `int MAX = 10;` declared in an interface?

- (a) `private static final`
- (b) `public final`
- (c) `public static final`
- (d) `protected static`

Q2. What happens?

```
interface X { int a; }
```

- (a) Compiles; `a` is `0`
- (b) Compile error: a final field must be initialized
- (c) Compiles with a warning
- (d) Compile error: interfaces cannot have fields

Q3. What happens?

```
interface Shape { double area(); }
class Circle implements Shape {
    double area() { return 3.14; }
}
```

- (a) Compiles

- (b) Compile error: weaker access privileges (the method must be `public`)
- (c) Runtime error
- (d) Compiles with a warning

Q4. Which modifier is NOT allowed on an abstract method in an interface?

- (a) `public`
- (b) `abstract`
- (c) `protected`
- (d) none of the above (omitting all modifiers)

Q5. Which Java version introduced `default` and `static` methods in interfaces?

- (a) Java 5
- (b) Java 7
- (c) Java 8
- (d) Java 11

Q6. Which Java version introduced `private` methods in interfaces?

- (a) Java 8
- (b) Java 9
- (c) Java 11
- (d) Java 17

Q7. What happens?

```
interface A { static void hi() { System.out.println("hi"); } }
class B implements A { }
B.hi();
```

- (a) Prints `hi`
- (b) Compile error: static interface methods are not inherited
- (c) Runtime error
- (d) Prints nothing

Q8. What happens?

```
interface Named { default String toString() { return "named"; } }
```

- (a) Compiles; all implementers print `named`

- (b) Compile error: a default method cannot override an `Object` method
- (c) Compiles with a warning
- (d) Runtime error

Q9. What happens?

```
interface A { default String hello() { return "A"; } }
interface B { default String hello() { return "B"; } }
class C implements A, B { }
```

- (a) Compiles; `hello()` returns `"A"`
- (b) Compiles; `hello()` returns `"B"`
- (c) Compile error: `C` must override `hello()`
- (d) Runtime error when called

Q10. What is printed?

```
class Base { public String hello() { return "Base"; } }
interface Greeter { default String hello() { return "Greeter"; } }
class Impl extends Base implements Greeter { }
System.out.println(new Impl().hello());
```

- (a) `Base`
- (b) `Greeter`
- (c) Compile error
- (d) `BaseGreeter`

Q11. What is printed?

```
interface Animal { default String sound() { return "..."; } }
interface Dog extends Animal { default String sound() { return "woof"; } }
class Puppy implements Animal, Dog { }
System.out.println(new Puppy().sound());
```

- (a) `...`
- (b) `woof`
- (c) Compile error
- (d) `...woof`

Q12. What happens?

```
interface A { int LIMIT = 10; }
interface B { int LIMIT = 20; }
class C implements A, B { int get() { return LIMIT; } }
```

- (a) Returns 10
- (b) Returns 20
- (c) Compile error: ambiguous reference
- (d) Returns 30

Q13. Which declaration is correct?

- (a) `class Duck extends Bird implements Flyer`
- (b) `class Duck implements Flyer extends Bird`
- (c) `class Duck extends Bird, Flyer`
- (d) `interface Duck implements Flyer`

Q14. Which of these is NOT allowed?

- (a) An interface extending two interfaces
- (b) A class extending one class and implementing two interfaces
- (c) An interface extending a class
- (d) An abstract class implementing an interface without implementing its methods

Q15. Which of these is a marker interface?

- (a) `Runnable`
- (b) `Serializable`
- (c) `Comparable`
- (d) `Iterator`

Q16. A functional interface has:

- (a) Exactly one method of any kind
- (b) Exactly one abstract method
- (c) At least one default method
- (d) No static methods

Q17. Is this a valid functional interface?

```
@FunctionalInterface
interface F {
    void a();
    boolean equals(Object o);
    default void b() { }
    static void c() { }
}
```

- (a) No: it has two abstract methods
- (b) Yes: `equals(Object)` does not count, and default and static methods do not count
- (c) No: functional interfaces cannot have static methods
- (d) No: functional interfaces cannot have default methods

Q18. What happens?

```
@FunctionalInterface
interface G { void a(); void b(); }
```

- (a) Compiles
- (b) Compile error from the annotation
- (c) Runtime error
- (d) `b()` is ignored

Q19. What happens?

```
sealed interface S permits X { }
final class X implements S { }
final class Y implements S { }
```

- (a) Compiles
- (b) Compile error: `Y` is not permitted
- (c) Compiles with a warning
- (d) Runtime error

Q20. What happens?

```
Runnable r = new Runnable();
```

- (a) Creates a `Runnable`

- (b) Compile error: an interface cannot be instantiated
- (c) `InstantiationException`
- (d) `r` is `null`

Q21. Which statement is true after Java 8?

- (a) An interface can have constructors
- (b) An abstract class can hold instance fields and constructors, an interface cannot
- (c) An interface can have instance fields
- (d) A class can extend several abstract classes

Q22. Which of these is a `default` method of a JDK interface?

- (a) `Collection.size()`
- (b) `Collection.stream()`
- (c) `List.get(int)`
- (d) `Runnable.run()`

Q23. A `private` interface method can be called from:

- (a) Any implementing class
- (b) Any class in the same package
- (c) The `default` and `static` methods of the same interface
- (d) Anywhere

Q24. What is printed?

```
interface A { default String who() { return "A"; } }
class B implements A {
    @Override public String who() { return A.super.who() + "B"; }
}
System.out.println(new B().who());
```

- (a) `A`
- (b) `B`
- (c) `AB`
- (d) Compile error

Q25. Which statement about interface constants is true?

- (a) They can be `private`

- (b) They are implicitly `public static final`
- (c) They can be reassigned in implementing classes
- (d) They need no initializer

Part B: Short Answer

Q26. What is an interface? List six things you can achieve with interfaces.

Q27. Why are interface variables always `public static final`, and interface methods `public abstract` by default? Give the design reasoning for each word.

Q28. Explain `extends`, `implements` and `interface` as keywords, including which combinations are allowed and how many parents each allows.

Q29. Why were default methods introduced? State their rules and restrictions.

Q30. Compare static and private methods in interfaces: purpose, visibility, inheritance, and version.

Q31. Summarize how interfaces evolved in Java 8, 9 and 17.

Q32. State the rules for resolving naming conflicts among interfaces (methods, constants, return types).

Q33. What is a marker interface? Give examples, explain how the tag takes effect, and compare it with an annotation.

Q34. What is a functional interface? What does `@FunctionalInterface` do, and what does and does not count as the single abstract method?

Q35. Compare an abstract class with an interface after Java 8, and give a rule for choosing.

Q36. What is a sealed interface and what does it give you?

Part C: Coding and Prediction

C1. Define a sealed interface `Shape` with records `Circle(double r)` and `Rectangle(double w, double h)`, and a method `area(Shape)` using a `switch` expression without `default`. Say what happens when you add a `Triangle` that is permitted but not handled.

C2. Write an interface `Cache<K, V>` with abstract `get` and `put`, a `default` method `getOrCompute(K key, Function<K, V> loader)`, a `static` factory `inMemory()`, and a `private` helper. Provide a simple implementation.

C3. Make this compile, with `describe()` returning both parts:

```
interface Camera { default String describe() { return "camera"; } }
interface Phone  { default String describe() { return "phone"; } }
class SmartPhone implements Camera, Phone { }
```

C4. Predict the output of `new D().m()`:

```
interface A { default String m() { return "A"; } }
interface B extends A { default String m() { return "B"; } }
interface C extends A { }
class D implements B, C { }
```

C5. Write a functional interface `Transformer<T>` with `T apply(T input)` and a default method `andThen(Transformer<T> next)`. Show an anonymous class and the lambda form, and chain two transformers on a string.

C6. Create a marker interface `Auditable`, two entities (one marked, one not) and a `save(Object)` method that logs only marked entities. Then write `archive(Auditable a)` and explain what the compiler does with `archive(new Plain())`.

C7. Which of these lines do not compile, and why?

```
interface I { int X; }                                          // L1
interface J { private void p(); }                               // L2
interface K { default void d(); }                               // L3
interface L { static void s() { } }
class M implements L { void t() { s(); } }                      // L4
interface N { default boolean equals(Object o) { return true; } }   // L5
```

Part D: Senior Level

S1. Why can default methods not be `final` or hold state? How does the JVM dispatch a call to a default method, and what does "maximally specific" mean in method selection?

S2. Explain source versus binary compatibility when evolving a published interface. What are the risks of adding an abstract method, a default method, or a static method?

S3. Explain JDK dynamic proxies (`Proxy.newProxyInstance`) versus CGLIB proxies. Why does the JDK proxy need an interface, and what are the limitations of each?

S4. When is an interface initialized? Predict the output below and explain the special case for default methods:

```
interface P { int V = init("P.V"); static int init(String s) {
System.out.println(s); return 1; } }
interface Q { int W = init("Q.W"); default void d() { } static int
init(String s) { System.out.println(s); return 2; } }
class Impl implements P, Q { }
new Impl();
System.out.println("created");
```

S5. Explain the rules and the error cases of `X.super.m()`: what must `X` be, and why can you not call a grandparent interface's default this way?

S6. Debate marker interface versus annotation for tagging domain classes. How would you design an `Auditable` tag, and what are the serialization-related risks of `Serializable`?

S7. Design a `Result<T>` type with a sealed interface and records. Explain exhaustiveness checking, and what happens at compile time and at runtime when a new permitted subtype is added later.

S8. Functional interface design: primitive specializations, checked exceptions in lambdas, wildcard parameters (`? super T`, `? extends R`), and why `@FunctionalInterface` is useful.

S9. Design the notification module of a service: interfaces, a skeletal abstract class, strategy selection, and Spring wiring. Justify each interface using ISP and DIP.

S10. A library must stay source- and binary-compatible for ten years while evolving. How do you use interfaces, abstract classes and default methods to achieve that?

S11. Compare `invokevirtual` with `invokeinterface`. Is calling through an interface slower in practice? Discuss itables, inline caches and megamorphic sites.

Answers: Part 3.4

Part A: Multiple Choice

Q	Answer	Explanation
1	(c)	Interface fields are always `public static final`.
2	(b)	A constant must be initialized where it is declared.

Q	Answer	Explanation
3	(b)	Interface methods are `public`; an implementing method with package access reduces visibility, which is illegal.
4	(c)	`protected` is not allowed (and neither are `private` or `final` on an abstract method). Writing no modifier is fine.
5	(c)	Java 8 (2014).
6	(b)	Java 9 (2017).
7	(b)	Static interface methods are not inherited, so `B.hi()` does not compile; write `A.hi()`.
8	(b)	A default method cannot override `equals`, `hashCode` or `toString`.
9	(c)	Two unrelated defaults with the same signature force the class to override.
10	(a) `Base`	A method inherited from a class always wins over an interface default.
11	(b) `woof`	`Dog` extends `Animal` and overrides `sound()`, so the more specific interface wins.
12	(c)	The simple name `LIMIT` is ambiguous; qualify as `A.LIMIT` or `B.LIMIT`.
13	(a)	`extends` comes first, then `implements`; a class cannot extend several classes; an interface uses `extends`.
14	(c)	An interface cannot extend a class.
15	(b)	`Serializable` has no members; it only tags a class.
16	(b)	Exactly one abstract method. Default, static and `Object`-method redeclarations do not count.
17	(b)	`equals(Object)` matches `Object`'s method and is not counted, so only `a()` is the single abstract method.

Q	Answer	Explanation
18	(b)	Two abstract methods violate the annotation, so the compiler reports an error.
19	(b)	`Y` is not in the `permits` list, so it cannot implement the sealed interface.
20	(b)	An interface cannot be instantiated with `new`; you need a class, an anonymous class or a lambda.
21	(b)	Only abstract classes can have instance fields and constructors. A class can extend only one class.
22	(b)	`Collection.stream()` is a `default` method added in Java 8. `size`, `get` and `run` are abstract.
23	(c)	Private interface methods are visible only inside the interface, for its default and static methods.
24	(c) `AB`	`A.super.who()` calls the interface's default (`"A"`) and the class appends `"B"`.
25	(b)	Interface constants are implicitly `public static final` and must be initialized.

Part B: Short Answer (model answers)

Q26. An interface is a reference type that declares **what** implementers must do (a contract), with no instance state. With it you get: (1) abstraction and a stable contract; (2) loose coupling, by programming to the type; (3) multiple inheritance of type; (4) runtime polymorphism; (5) testability and dependency injection (fakes, mocks); (6) capabilities shared by unrelated classes (`Comparable`, `Runnable`); (7) lambdas and callbacks through functional interfaces; (8) safe API evolution with default methods.

Q27. *Fields:* **public**, because the contract must be visible to every implementer and caller in any package; **static**, because an interface has no instances and no state, and instance fields would bring back multiple inheritance of state and its diamond problem; **final**, because the value is shared by everyone, and a reassignable shared value would be global mutable state that changes the contract for all. *Methods:* **public**, because they are the contract that callers use and implementers cannot reduce visibility; **abstract**, because an interface specifies the *what* and leaves the *how* to the implementers, which also permits many unrelated implementations.

Q28. `interface` declares the type. `extends` between classes gives one parent class (implementation and state); `extends` between interfaces allows several parent interfaces. `implements` lets a class accept any number of interface contracts. A class that uses both writes `extends` first, then `implements`. An interface cannot extend or implement a class, and a class cannot `extend` an interface.

Q29. To add behavior to existing interfaces without breaking their implementers, mainly so `Collection.stream()`, `List.sort` and `Iterable.forEach` could be added in Java 8, and to build combinators such as `Comparator.reversed()` and `Predicate.and()`. Rules: declared with `default` and a body; implicitly public; inherited and overridable by implementers; may call `this` methods but has no instance state; cannot be `static`, `final`, `abstract` or `synchronized`; cannot override `Object` methods; use `X.super.m()` to call a specific parent's version.

Q30. A **static** interface method (Java 8) belongs to the interface itself, is public by default, is **not inherited**, is called `Interface.m()`, and is used for factories and helpers (`List.of`). A **private** method (Java 9) is visible only inside the interface, may be instance or static, and exists to share code among default and static methods without exposing it.

Q31. Before Java 8: constants and abstract methods only. **Java 8:** `default` and `static` methods, and functional interfaces with lambdas. **Java 9:** `private` and `private static` methods. **Java 17:** `sealed` interfaces with `permits`, giving closed hierarchies. (Records and enums can implement interfaces, and pattern matching `switch` over sealed interfaces arrived through Java 21.)

Q32. The same abstract method from two interfaces is satisfied by one implementation; incompatible return types are an error; different parameters are overloads. For conflicting defaults: (1) a class method wins; (2) otherwise the more specific interface wins; (3) otherwise the class must override and may use `A.super.m()`. Static interface methods never conflict (not inherited). Constants with the same name are ambiguous and must be qualified (`A.X`). Defaults may not redefine `Object` methods.

Q33. A marker interface is an empty interface used to tag a class: `Serializable`, `Cloneable`, `RandomAccess`. The tag works because runtime code checks `instanceof` (the serialization machinery throws `NotSerializableException` for untagged classes; `Collections.binarySearch` picks an algorithm if the list is `RandomAccess`). Compared with an annotation: a marker interface defines a **type** (usable in signatures, checked at compile time) and is inherited, but applies only to classes and carries no data; an annotation can mark any element, carries parameters, but defines no type.

Q34. A functional interface is a SAM interface that can be the target of a lambda or method reference. `@FunctionalInterface` is optional, and makes the compiler verify the single abstract method. Not counted: default and static methods, and abstract redeclarations of `Object` methods such as `equals`. It may extend another interface as long as only one abstract method remains. Examples: `Runnable`, `Comparator`, `Function`.

Q35. An abstract class can have state, constructors, any access level, `final` methods and `protected` hooks, and is limited to one per class. An interface has no state or constructors, only public contract methods plus defaults, statics and private helpers, and a class may implement many; only interfaces can be lambda targets. Choose an interface for contracts, capabilities and lambdas; an abstract class for closely related types that share state or an algorithm skeleton; and often both, as interface plus skeletal abstract class.

Q36. A sealed interface restricts its implementers to the types in a `permits` list (each `final`, `sealed` or `non-sealed`; records are final). It gives a closed, known family, so a `switch` over it can be exhaustive without `default`, and adding a new subtype makes the compiler flag every switch that does not handle it.

Part C: Coding and Prediction (model solutions)

C1. Sealed shapes

```
sealed interface Shape permits Circle, Rectangle { }
record Circle(double r) implements Shape { }
record Rectangle(double w, double h) implements Shape { }

static double area(Shape s) {
    return switch (s) {
        case Circle c    -> Math.PI * c.r() * c.r();
        case Rectangle r -> r.w() * r.h();
    };                                  // no default: the compiler knows
the permitted set
}
```

If you add `Triangle` to `permits` and declare `record Triangle(...) implements Shape`, `area` no longer compiles ("the switch expression does not cover all possible input values") until you add a `case Triangle`. If you declare a `Triangle` that is **not** in `permits`, the declaration itself fails.

C2. A cache interface

```
interface Cache<K, V> {
    V get(K key);
    void put(K key, V value);

    default V getOrCompute(K key, Function<K, V> loader) {
        V v = get(key);
        if (v == null) { v = load(loader, key); put(key, v); }
        return v;
    }
    private V load(Function<K, V> loader, K key) {            // private
helper: hidden from implementers
        return Objects.requireNonNull(loader.apply(key), "loader returned
null");
    }
    static <K, V> Cache<K, V> inMemory() { return new MapCache<>(); }   //
factory next to the type
}
class MapCache<K, V> implements Cache<K, V> {
    private final Map<K, V> map = new ConcurrentHashMap<>();
    public V get(K key)            { return map.get(key); }
    public void put(K key, V value){ map.put(key, value); }
}
```

The check-then-put in `getOrCompute` is not atomic; for concurrent use, `ConcurrentHashMap.computeIfAbsent` is the right primitive.

C3. Resolving the conflict

```
class SmartPhone implements Camera, Phone {
    @Override public String describe() {
        return Camera.super.describe() + " + " + Phone.super.describe();
// "camera + phone"
    }
}
```

C4. Output: `B`. `D` inherits `m()` from `B` (which overrides `A.m`) and, through `C`, the inherited `A.m`. Because `B.m` **overrides** `A.m`, `A.m` is not a candidate, so the most specific declaration, `B.m`, wins with no conflict.

C5. Functional `Transformer`

```
@FunctionalInterface
interface Transformer<T> {
    T apply(T input);
    default Transformer<T> andThen(Transformer<T> next) { return x ->
next.apply(apply(x)); }
}

Transformer<String> trim = new Transformer<>() {              // anonymous
class
    @Override public String apply(String s) { return s.trim(); }
};
Transformer<String> upper = s -> s.toUpperCase();             // lambda
System.out.println(trim.andThen(upper).apply("  hello "));    // HELLO
```

C6. Marker interface

```
interface Auditable { }
class Invoice implements Auditable { }
class Plain { }

static void save(Object entity) {
    if (entity instanceof Auditable) System.out.println("audit: " +
entity.getClass().getSimpleName());
}
static void archive(Auditable a) { }

save(new Invoice());      // audit: Invoice
save(new Plain());        // nothing
archive(new Invoice());   // OK
// archive(new Plain());  // COMPILE ERROR: Plain cannot be converted to
Auditable
```

The compile error is the advantage of a marker interface over an annotation: the tag is part of the type system.

C7. All five lines fail

- L1: `int X;` must be initialized.
- L2: a `private` method needs a body (private abstract methods are meaningless).
- L3: a `default` method must have a body.
- L4: static interface methods are not inherited, so `s()` is not found inside `M`; write `L.s()`.

- L5: a default method cannot override `equals`.

Part D: Senior Level (model answers)

S1. A default is a **fallback** that implementers may replace, and conflict resolution depends on classes and sub-interfaces being able to override it, so `final` contradicts its purpose. Interfaces have no instance fields by design, to avoid inheriting state from several parents, so a default can use only the interface's other methods. For dispatch, `invokeinterface` (or `invokevirtual` on a class-typed reference) uses **method selection** (JVMS 5.4.6): look in the receiver's class and its superclasses; if nothing is found, collect the **maximally specific** superinterface methods, meaning those not overridden by a more specific interface. If exactly one is non-abstract it is selected. If two unrelated defaults remain (possible when interfaces are recompiled separately), the JVM throws `IncompatibleClassChangeError`; if only abstract ones remain, `AbstractMethodError`.

S2. *Adding an abstract method* breaks **source** compatibility (every implementer fails to compile) and **binary** compatibility (callers get `AbstractMethodError`). *Adding a default method* is normally compatible, but can create a conflict with a method of the same signature in an unrelated interface that an implementer also implements (compile error on recompilation; `IncompatibleClassChangeError` at runtime if only one side was recompiled), clash with an existing class method of incompatible return type or weaker access, or silently lose to an existing class method of the same name. *Adding a static method* is safe, because it is not inherited. Removing or renaming members breaks both. Practices: treat published interfaces as frozen; evolve with defaults; introduce a new sub-interface for larger changes; maintain skeletal abstract classes that absorb new methods; deprecate before removal (`@Deprecated(forRemoval = true)`); and check compatibility automatically with tools such as japicmp or revapi.

S3. `Proxy.newProxyInstance(loader, new Class[]{Svc.class}, handler)` generates a class at runtime that **extends `java.lang.reflect.Proxy`** and implements the listed interfaces; every interface method call (and `equals`, `hashCode`, `toString`) is routed to `InvocationHandler.invoke(proxy, method, args)`. It needs interfaces because the generated class already uses its one superclass slot for `Proxy`, and it can only intercept interface methods. A proxy is not an instance of the target class, so injecting by class type gives a `ClassCastException`. **CGLIB or ByteBuddy** generate a **subclass** of the target class, so they can proxy classes without interfaces, at the cost of the limits in 3.3 S6: `final` classes and methods and `private` methods cannot be intercepted, and a usable

constructor is needed (Spring uses Objenesis). Both approaches miss **self-invocation** (a call through `this`). Spring picks a JDK proxy when an interface exists and class-proxying is off; Spring Boot sets `proxyTargetClass=true` by default, so CGLIB is used.

S4. Output:

```
Q.W
created
```

Initializing a class does **not** initialize its superinterfaces, **except** those that declare at least one **default** method (JLS 12.4.2). `Q` declares `d()` as a default, so it is initialized when `Impl` is, which runs `init("Q.W")`. `P` has no default methods and nobody reads `P.V`, so `P` is never initialized and prints nothing.

S5. `X.super.m()` is legal only when `X` is a **direct superinterface** of the class or interface being declared, and it cannot be used in a static context. It is a compile error if **another direct superinterface (or the superclass) overrides** that method, because then the call would bypass the more specific version. So if `class C implements A, B` and `B extends A` overrides `m()`, then `A.super.m()` is rejected; you may call `B.super.m()`. You also cannot reach a grandparent: `Z.super.m()` fails when `Z` is not a direct parent. The rule keeps `super` calls consistent with Rule 2 (the more specific interface wins).

S6. A marker interface creates a **type**: methods can demand it (`archive(Auditable a)`), generic bounds can use it (`<T extends Auditable>`), and misuse fails at compile time. An annotation can tag methods, fields and packages, and carry parameters (`@Audited(level = HIGH)`), but is invisible to the type system and needs reflection or processing. Choose a marker interface for "this class has this property, and APIs must require it", and an annotation for metadata that frameworks read. Prefer a real method (`auditId()`) over a pure marker if the tag implies behavior. `Serializable` risks: it silently makes the class's internal representation part of its public API (`serialVersionUID` and field changes break compatibility); it bypasses constructors and invariants; every subclass inherits it; and deserialization of untrusted data is a major vulnerability (gadget chains), mitigated by `ObjectInputFilter` or by avoiding Java serialization in favor of JSON or Protobuf.

S7.

```
sealed interface Result<T> permits Result.Ok, Result.Err {
    record Ok<T>(T value) implements Result<T> { }
    record Err<T>(String message) implements Result<T> { }
}
static <T> String show(Result<T> r) {
    return switch (r) {
        case Result.Ok<T> ok   -> "ok " + ok.value();
        case Result.Err<T> err -> "error " + err.message();
    };
}
```

The compiler checks that the `switch` covers every permitted subtype. **When a new subtype (say `Pending`) is added to `permits` and the project is recompiled,** every non-exhaustive `switch` becomes a **compile error**, which is the safety benefit. If the library is changed and the **client is not recompiled**, the old compiled `switch` meets an unknown subtype at **runtime** and throws `MatchException` (earlier JDKs: `IncompatibleClassChangeError`). Avoid sealed types for **open** extension points that outsiders must implement, and note that adding a subtype is a breaking change for consumers of your library.

S8. Use **primitive specializations** (`IntFunction`, `ToLongFunction`, `IntBinaryOperator`) in hot paths to avoid boxing. **Checked exceptions:** the SAM method does not declare them, so a lambda cannot throw them; define your own `interface ThrowingFunction<T, R, E extends Exception> { R apply(T t) throws E; }` and wrap at the boundary. **Wildcards** follow PECS: parameters that **consume** use `? super T`, ones that **produce** use `? extends R`, as in `Stream.map(Function<? super T, ? extends R>)`, which lets callers pass `Function<Object, String>` for a `Stream<Integer>`. `@FunctionalInterface` protects you: if someone later adds a second abstract method, the compiler fails the interface instead of breaking every lambda silently, and it documents the intent. Also avoid overloads that accept different functional interfaces with the same arity (`submit(Runnable)` vs `submit(Callable)` can make lambdas ambiguous).

S9.

```
interface NotificationChannel {                          // ISP: one small,
focused contract
    Channel channel();
    void send(Notification n);
}
```

```
abstract class AbstractChannel implements NotificationChannel {   //
skeleton: template method
    @Override public final void send(Notification n) {
        validate(n);
        doSend(n);
        audit(n);
    }
    protected void validate(Notification n) { Objects.requireNonNull(n); }
    protected abstract void doSend(Notification n);
    private void audit(Notification n) { /* common logging or metrics */ }
}
@Component class EmailChannel extends AbstractChannel { public Channel
channel() { return Channel.EMAIL; } protected void doSend(Notification n) {
/* SMTP */ } }
@Component class SmsChannel   extends AbstractChannel { public Channel
channel() { return Channel.SMS;   } protected void doSend(Notification n) {
/* gateway */ } }

@Service
class NotificationService {                              // DIP: depends on
the abstraction only
    private final Map<Channel, NotificationChannel> byChannel;
    NotificationService(List<NotificationChannel> channels) {          //
Spring injects every implementation
        this.byChannel =
channels.stream().collect(Collectors.toMap(NotificationChannel::channel,
Function.identity()));
    }
    void notify(Channel c, Notification n) { byChannel.get(c).send(n); }   //
strategy selection by key
}
```

Justification: the **interface** is the stable contract (DIP) that the service and tests use; the **abstract skeleton** removes duplicated validation and auditing and fixes the order of steps; a new `PushChannel` is just another bean, so the service is **closed for modification, open for extension**; tests inject a fake channel; and a read-only `NotificationQuery` interface could be split out so reporting code cannot send messages (ISP).

S10. Separate **API** types that only you implement from **SPI** types that users implement. For API interfaces, prevent outside implementations (package-private constructors, `sealed`), so you may add abstract methods freely. For SPI interfaces, treat them as **frozen**: evolve with `default` methods, offer a **skeletal abstract class** so new methods

can be given sensible implementations there, or create `FooV2 extends Foo` for bigger changes. Prefer returning interface types and using static factories so you can change implementations; keep value types `final` or `sealed`; avoid public constants and exposed implementation classes; use modules (`exports`) to shrink the public surface; follow semantic versioning with `@since` and `@Deprecated(forRemoval = true)` and a grace period; and enforce compatibility in CI with japicmp or revapi.

S11. For a class-typed receiver, every overridable method has a **fixed index** in the class's vtable, shared by all subclasses, so `invokevirtual` is one indexed load. For an interface, a class can implement the interface at arbitrary places in its hierarchy, so there is no single fixed index; `invokeinterface` consults an **itable** (find the interface's section, then the method offset), a few extra steps. In compiled code the difference mostly disappears: **class hierarchy analysis** inlines an interface method if it has a single implementer, and **inline caches** handle monomorphic and bimorphic sites with a simple type check, equally for both kinds of call. A **megamorphic** site is slower for both, and a little slower through an interface. Benchmark with JMH before letting this influence design; programming to interfaces is almost never the bottleneck, and megamorphic sites matter far more than the invoke instruction.

End of Part 3.4. Next is **Part 3.5: Object contracts and value design** (the `Object` class methods in full, the `equals()` / `hashCode()` contract, `Comparable` vs `Comparator`, immutable class design, records, enums, sealed classes and a closer look at pass-by-value). Tell me when to start, or flag anything here you want expanded first.